Chapter 4
Keys, Addresses

Course Notes

CSE0469 Blockchain: Special Topics in SW Engineering

Yusuf Altunel

Based on Mastering Bitcoin 2e; Andreas M. Antonopoulos; O'Reilly, 2017

Content



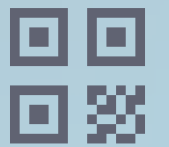
Introduction

Public Key Cryptography & Cryptocurrency

Private and Public Keys

Elliptic Curve Cryptography

Generating a Public Key



Bitcoin Addresses

Base58 & Base58Check Encoding

Key Formats



Implementation

Implementing Keys and Addresses in Python



Advanced Keys and Addresses

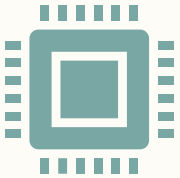
Encrypted Private Keys

Pay-to-Script Hash (P2SH) & Multisig Addresses

Vanity Addresses

Paper Wallets

Introduction



cryptography

a branch of mathematics

used extensively in computer security.



Greek

«secret writing» in Greek,



encryption

the science of cryptography

encompasses more than just «secret writing»

Introduction



digital signature

to prove knowledge of a secret

without revealing that secret



digital fingerprint

Proves the authenticity of data

Wallet & Keys



Ownership

Bitcoin ownership is established through

digital keys, *bitcoin addresses*, and *digital signatures*.

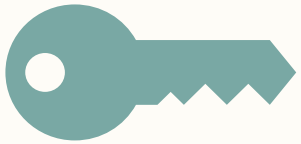


The digital keys

independent of the bitcoin protocol

maintained by the user's wallet software

Wallet & Keys

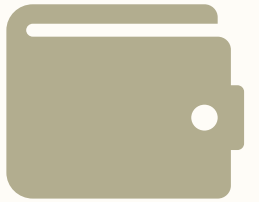


Keys

for decentralized trust and control

ownership attestation

the cryptographic-proof security model



Wallet

Keyse are created and stored by users

in a file

or simple database

called a *wallet*.

Validity of Bitcoin Ownership



Transaction validity

a valid digital signature

to be included in the transaction



Signature

Proves the ownership

without revealing the private key



Control

Signature can only be generated

if the private key is knwon



Ownership

anyone with a copy of that key

has control of the bitcoin

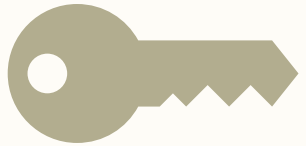
Validity of Bitcoin Ownership



Witness

A term used in cryptography

The digital signature used to spend funds



True ownership

The witness data in a bitcoin transaction testifies to the true ownership of the funds being spent

Private-Public Key Pairs



Key pairs

Keys come in pairs



Composition

a private (secret) key

a public key



Public key

similar to a bank account number



Private key

similar to the secret PIN, or signature on a check

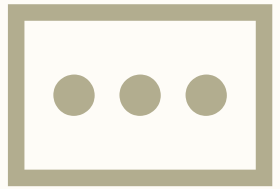
provides control over the account.

Private-Public Key Pairs



Visibility

digital keys are very rarely seen by the users of bitcoin.



Storage

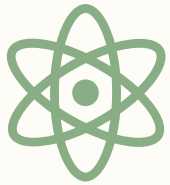
they are stored inside the wallet file managed by the bitcoin wallet software.

Public Key Cryptography & Cryptocurrency



Public key cryptography

was invented in the 1970s



Mathematics

a mathematical foundation

for computer and information security



Later discoveries

such as prime number exponentiation

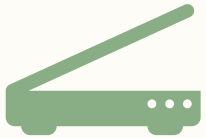
elliptic curve multiplication

Public Key Cryptography & Cryptocurrency



Irreversability

Ecliptic curve functions are practically irreversible



Meaning

easy to calculate in one direction

infeasible to calculate in the opposite direction



Cryptography

Enables the creation of digital secrets

unforgeable digital signatures.

Key Pairs



Key pair in Bitcoin

The key pair consists of a private key

derived from it—a unique public key.



The public key

is used to receive funds



The private key

is used to sign transactions to spend the funds

Key Pairs



Signatures on Messages

There is a mathematical relationship

between the public and the private key



Signatures from private key

the private key is used

to generate signatures on messages.



Validation of Signature

This signature can be validated against the public key

without revealing the private key.

Execution



Spending bitcoin

the current bitcoin owner presents

public key and a signature in a transaction



Signature

Each time is different

but created from the same private key.

Execution



Verification

everyone in the bitcoin network

can verify and accept the transaction as valid

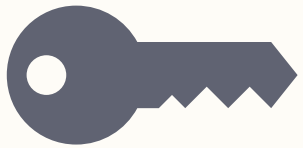


Shows

the person transferring the bitcoin

owned them at the time of the transfer.

Keys in Wallet Implementations



Usual Storage Strategy

the private and public keys

are stored together as a key pair for convenience.



Production of Public Key

public key can be calculated from the private key

so storing only the private key is also possible.

Private and Public Keys



Private key - k

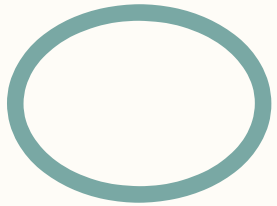
The private key (k) is a number, usually picked at random.



Public key - K

Generated from the private key

Private and Public Keys



Elliptic curve multiplication

a one-way cryptographic function



Bitcoin address - A

Generated from the public key (K),

Using a one-way cryptographic hash function

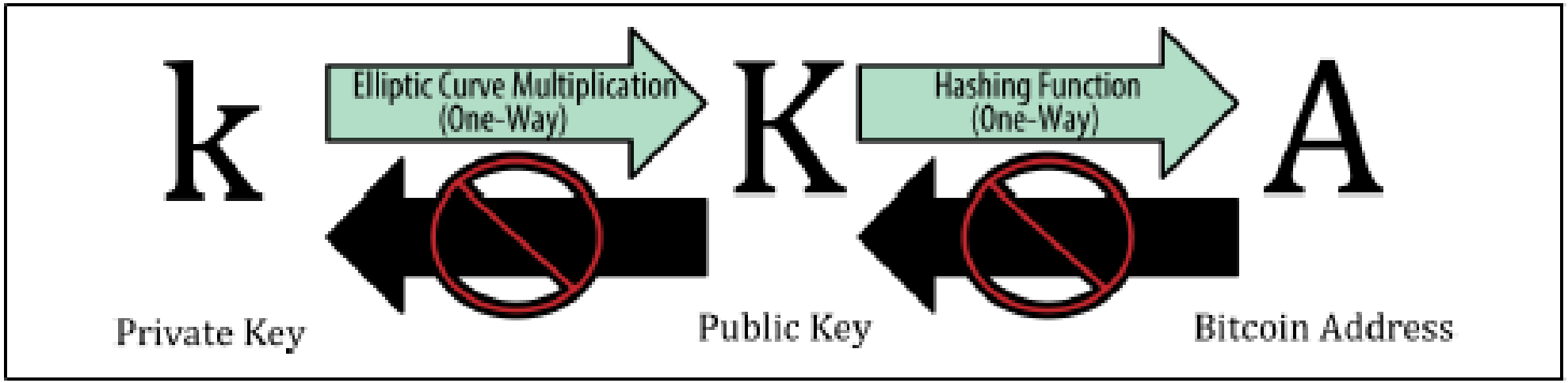


Figure 4-1. Private key, public key, and bitcoin address

Why Use Asymmetric Cryptography?



Asymmetric cryptography

Public/Private Keys



Verification

anyone can verify every signature on every transaction



Authority to produce

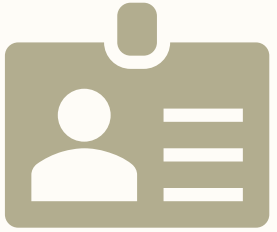
only the owners of private keys can produce valid signatures

Why Use Asymmetric Cryptography?



Verification

Public key and transaction fingerprint



Public

Anyone can verify

every signature

on every transaction

Private Keys



Random

A private key is simply a number

picked at random.



Fund control and ownership

the private key is the root of user control over all funds

associated with the corresponding bitcoin address.



Spending

The private key is used to create signatures

required to spend bitcoin

by proving ownership of funds used in a transaction.

Private Keys



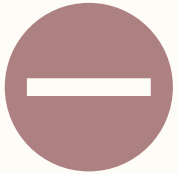
The private key must remain secret

prevent the third parties from giving control
over the bitcoin secured by that key.



Backup

The private key must be backed up and protected
against accidental loss,



No recovery

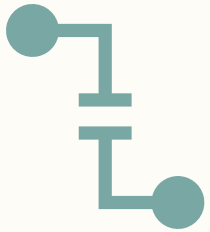
it cannot be recovered
the funds secured by it are forever lost, too.



The bitcoin private key is just a number. You can pick your private keys randomly using just a coin, pencil, and paper: toss a coin 256 times and you have the binary digits of a random private key you can use in a bitcoin wallet. The public key can then be generated from the private key.

101001…

Generating a private key from a random number



Randomness

find a secure source of entropy, or randomness

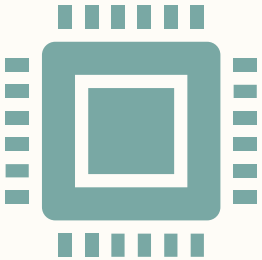
“Pick a number between 1 and 2^{256}.”



Rule

Method should not predictable or repeatable

Generating a private key from a random number



Bitcoin software's generator

uses the underlying operating system's random number generators



the private key

any number between 1 and n - 1, where n is a constant

$n = 1.158 * 10^{77}$, slightly less than 2^{256}

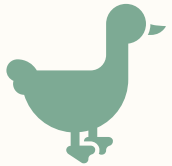
Algorithm



Key Production

randomly pick a 256-bit number

check that it is less than n - 1.



Method

feeding a larger string of random bits

into the SHA256 hash algorithm



SHA256

produce a 256-bit number.

Algorithm



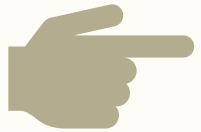
Strings

collected from a cryptographically secure source of randomness,



Check

If the result is less than n - 1, we have a suitable private key.



Otherwise

we simply try again with another random number.



Do not write your own code to create a random number or use a "simple" random number generator offered by your programming language. Use a cryptographically secure pseudorandom number generator (CSPRNG) with a seed from a source of sufficient entropy. Study the documentation of the random number generator library you choose to make sure it is cryptographically secure. Correct implementation of the CSPRNG is critical to the security of the keys.



The size of bitcoin's private key space, (2^{256}) is an unfathomably large number. It is approximately 10^{77} in decimal. For comparison, the visible universe is estimated to contain 10^{80} atoms.

New Key with Bitcoin Core Client



Generate a new key

use the *getnewad*dress command.



Displays

For security reasons

it displays the public key only



not

the private key.

```
$ bitcoin-cli getnewaddress
1J7mdg5rbQyUHENYdx39WVWK7fsLpEoXZy
$ bitcoin-cli dumpprivkey 1J7mdg5rbQyUHENYdx39WVWK7fsLpEoXZy
KxFC1jmwwCoACiCAWZ3eXa96mBM6tb3TYzGmf6YwgdGWZgawvrtJ
```

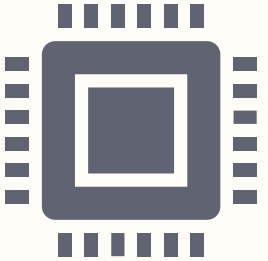
New Key with Bitcoin Core Client



Expose the private key

use the *dumpprivkey* command

$ bitcoin-cli getnewaddress
1J7mdg5rbQyUHENYdx39WVWK7fsLpEoXZy
$ bitcoin-cli dumpprivkey 1J7mdg5rbQyUHENYdx39WVWK7fsLpEoXZy
KxFC1jmwwCoACiCAWZ3eXa96mBM6tb3TYzGmf6YwgdGWZgawvrtJ



Wallet Import Format (WIF)

shows the private key

in a Base58 checksum-encoded format

Revealing the Private Key



***dumpprivkey* command**

opens the wallet and extracts the private key

generated by the get *newaddress* command.



No Private key from Public key

It is not possible for *bitcoind*

to know the private key from the public key



unless

they are both stored in the wallet

Bitcoin Explorer for Private Keys

ec-new, and *ec-to-wif*

- Used in the Bitcoin Explorer command-line tool

to generate and display private keys

- with the commands seed

```
$ bx seed | bx ec-new | bx ec-to-wif
5J3mBbAH58CpQ3Y5RNJpUKPE62SQ5tfcvU2JpbnkeyhfsYB1Jcn
```

Public Keys



Calculation

from the private key using elliptic curve multiplication,

which is irreversible



Formula

$K = k * G$



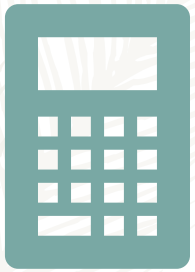
Legend

k is the private key

G is a constant point called the *generator point*

K is the resulting public key.

Public Keys



The reverse operation

known as “finding the discrete logarithm”

calculating *k* if you know *K*



Difficulty

is as difficult as trying all possible values of *k*

i.e., a brute-force search.

Elliptic Curve Multiplication



“trap door” function

it is easy to do in one direction (multiplication)

impossible to do in the reverse direction (division).



The owner of the private key

can easily create the public key

and then share it with the world

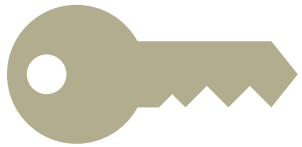
Elliptic Curve Multiplication



Protection

knowing just the public key

no one can reverse the function



Mathematical trick

becomes the basis

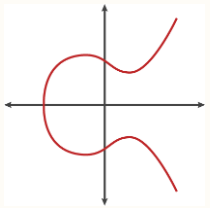
- for unforgeable and secure digital signatures
- that prove ownership of bitcoin funds.

Elliptic Curve Cryptography



Elliptic curve cryptography

type of asymmetric
public key cryptography



based on

the discrete logarithm problem



expressed

by addition and multiplication
on the points of an elliptic curve

Standard: secp256k1

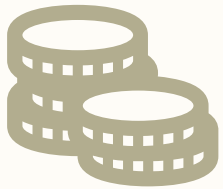
Established by	NIST
	the National Institute of Standards and Technology
Secp256k1	Standard
	produces secp256k1 elliptic curve
Elliptic curve	set of mathematical constants

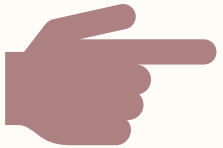
Formulization

$$y^2 \mod p = \left(x^3 + 7\right) \mod p$$

$$y^2 = \left(x^3 + 7\right) \text{ over } \left(\mathbb{F}_p\right)$$







The *mod p*

modulo prime number p

Curve indicates

over a finite field of

prime order *p*

P is Prime

a very large prime number

$2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1$

Formulization

$$y^2 = (x^3 + 7) \text{ over } (\mathbb{F}_p)$$

$$y^2 \mod p = (x^3 + 7) \mod p$$





example

the following is

a point p with coordinates (x,y)

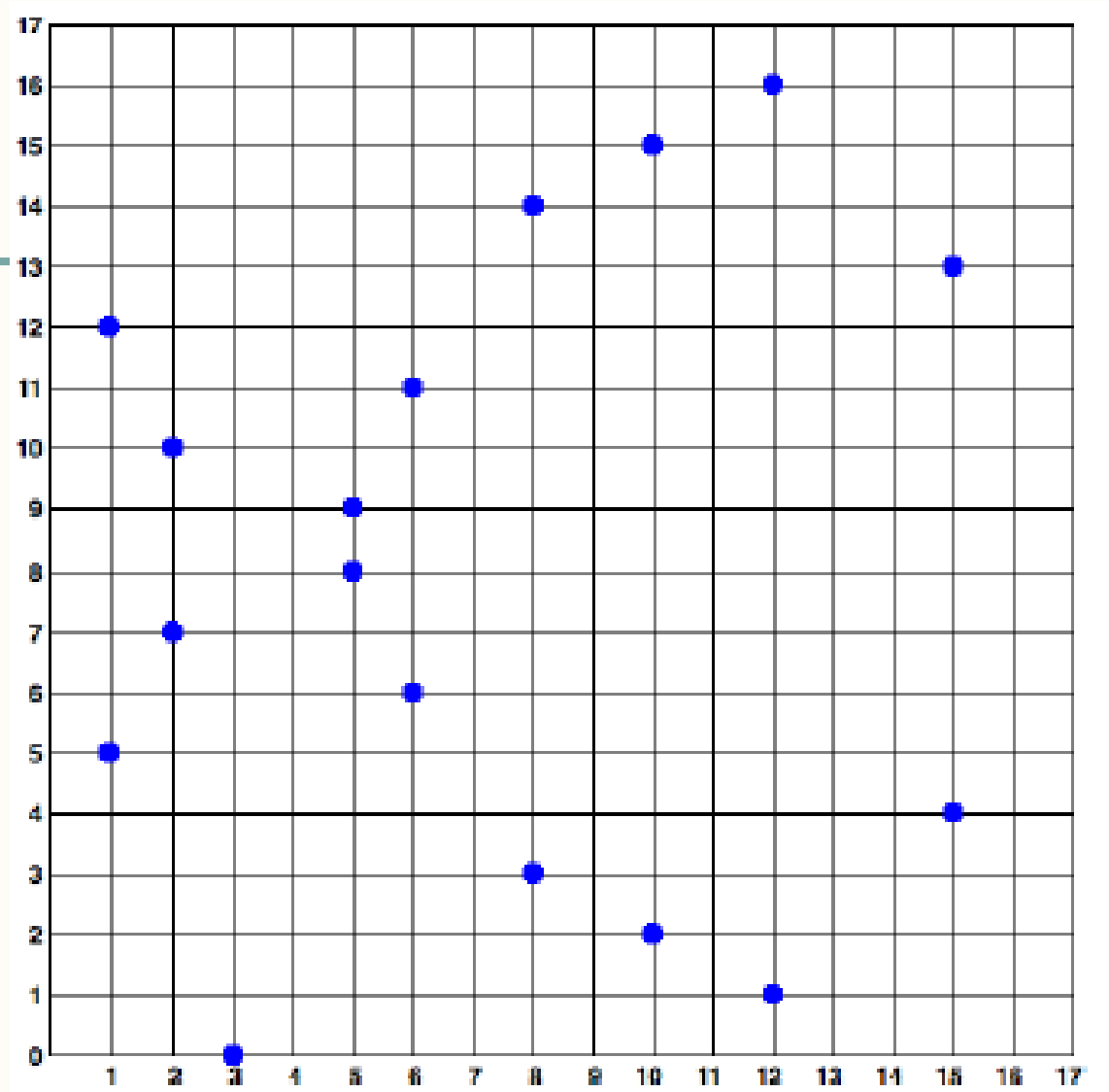
is a point

on the secp256k1 curve

```
P =
(55066263022773436695787188951685343262506034537775941755001873603891167292 40,
32670510020758816978083085130507043184471273380659243275938904335757337482424)
```

Elliptic curve cryptography

$$y^2 \mod p = (x^3 + 7) \mod p$$



visualizing an elliptic curve over F(p), with p=17

Generating a Public Key



Private key k

Start with private key

a randomly generated number ***k***



Multiply *k*

with a point on the curve ***G***



generator point *G*

a predetermined point

on the curve

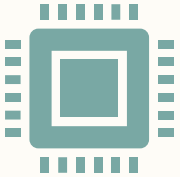


another point

somewhere else on the curve

public key ***K***

Generating a Public Key



G

The generator point

part of the secp256k1 standard



Always the same

for all keys in bitcoin



Formula

$K = k * G$

Generating a Public Key



Formula

$K = k * G$



k

the private key



G

the generator point



K

the resulting public key

a point on the curve

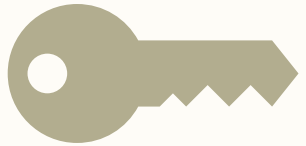
Safety of Public Keys



The same generator point

the generator point is always the same

for all bitcoin users



Private key to the same public key

a private key *k* multiplied with *G*

will always result in the same public key *K*

Safety of Public Keys



Fixed relationship

The relationship between *k* and *K* is fixed



One direction of calculation

Can only be calculated in one direction from *k* to *K*



Sharing public key is safe

bitcoin address (derived from *K*) can be shared with anyone and does not reveal the user's private key (*k*).

Implementation





Public key *K*

is as a point K = (x, y)

Public Key (K) Generation

take the private key *k* generated previously

multiply it with the generator point G

```
K = (x, y)

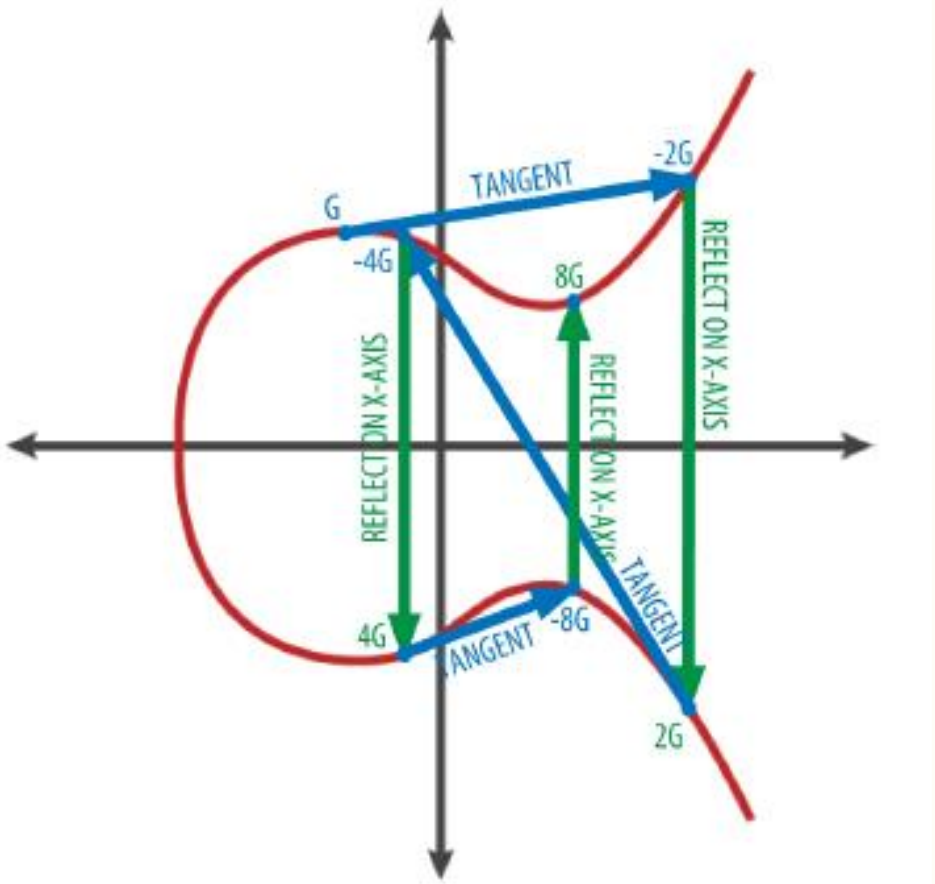
where,

x = F028892BAD7ED57D2FB57BF33081D5CFCF6F9ED3D3D7F159C2E2FFF579DC341A
y = 07CF33DA18BD734C600B96A72BBC4749D5141C90EC8AC328AE52DDFE2E505BDB
```

```
K = 1E99423A4ED27608A15A2616A2B0E9E52CED330AC530EDCC32C8FFC6A526AEDD * G
```



Most bitcoin implementations use the OpenSSL cryptographic library to do the elliptic curve math. For example, to derive the public key, the function `EC_POINT_mul()` is used.



In elliptic curves, adding a point to itself is the equivalent of

1. drawing a tangent line on the point
2. finding where it intersects the curve again
3. then reflecting that point on the x-axis

***Figure 4-4**. Elliptic curve cryptography: visualizing the multiplication of a point G by an integer k on an elliptic curve*

Bitcoin Addresses



Address

a string of digits and characters



to share with anyone

who wants to send you money.



Production

from public keys

Bitcoin Addresses



Composition

a string of numbers and letters,

beginnis with “1.”

1J7mdg5rbQyUHENYdx39WVWK7fsLpEoXZy



Recipient

The bitcoin address appears in a transaction

as the “**recipient**” of the funds

Bitcoin Address



The bitcoin address

is derived from the public key

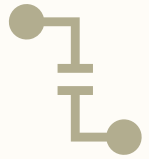
through the use of one-way cryptographic hashing.



“hashing algorithm”

or simply “hash algorithm”

is a one-way function



Usage

produces a fingerprint:

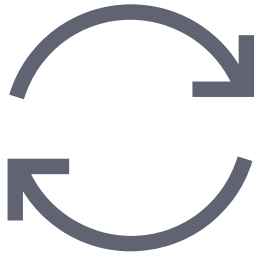
“hash” of an arbitrary-sized input.

Bitcoin Address



SHA256

the **S**ecure **H**ash **A**lgorithm



RIPEMD160

the **R**ACE **I**ntegrity **P**rimitives **E**valuation **M**essage **D**igest

Calculation



K

the public key



A

the resulting bitcoin address



Steps

SHA256 hash of K

RIPEMD160 hash of the result



A

= *RIPEMD*160 *SHA*256 *K*

160-bit/**20**-byte number

Bitcoin Address vs Public Key



Bitcoin Address

is *not* the same

as a public key



Bitcoin addresses

are derived from a public key

using a one-way function

Bitcoin Address vs Public Key



“Base58Check”

encoded with 58 characters

a checksum



Checksum

help human readability

avoid ambiguity



protect against errors

in address transcription and entry

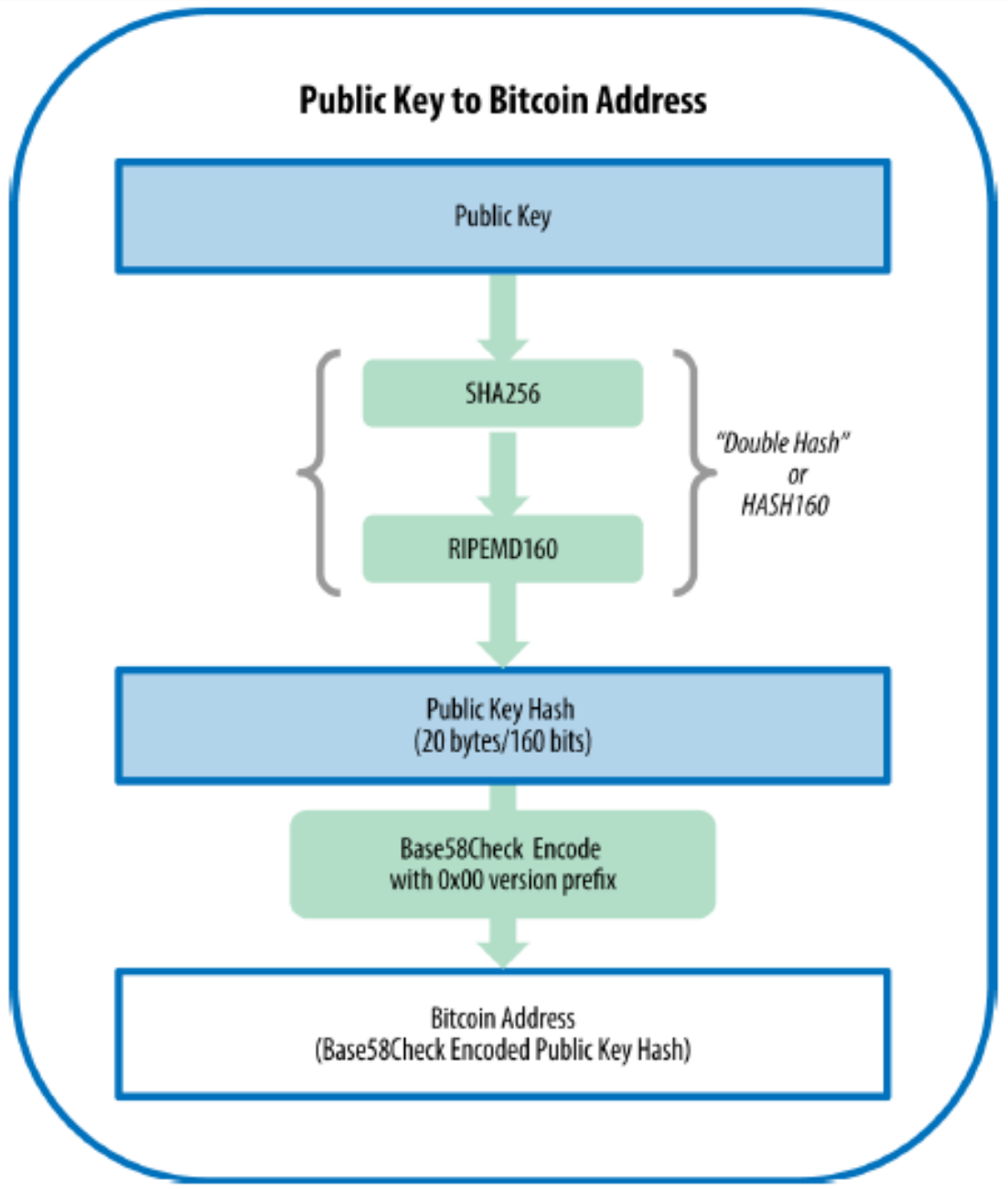


Figure 4-5. Public key to bitcoin address: conversion of a public key into a bitcoin address

Decimal vs Hex Encoding

0.0

Decimal system

uses the 10 numerals

0 through 9

Hexadecimal system

uses 16: 10 numerals

with the letters A through F.

Hex vs Base64 Encoding



Hexadecimal shorter

A number in hexadecimal is shorter than decimal representation.



Base64 representation

Shorter than Hexadecimal

Base64 Encoding

52 letter	10 numerals	2 characters:
26 lowercase letters 26 capital letters	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	«+», «/»

Base64 vs Base58



Base58 vs Base64

Base58 is a subset of Base64

using upper- and lowercase letters and numbers



Omit

some characters

frequently mistaken

Base64 vs Base58

Omitted	Base58 alphabet
• 0: number zero • O: capital o • l: lower L • I: capital i • the symbols “+” and “/”	• 123456789 • ABCDEFGHJKLMNPQRSTUVWXYZ • abcdefghijkmnopqrstuvwxyz

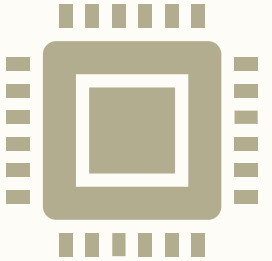
Base64 vs Base58



Base64

used to add

binary attachments to email



Base58

is a textbased binary-encoding format

used in cryptocurrencies

Base58 to Base58Check



“version byte”

a prefix added to the data,

to easily identify the encoded type



0x00 in hex

a bitcoin address

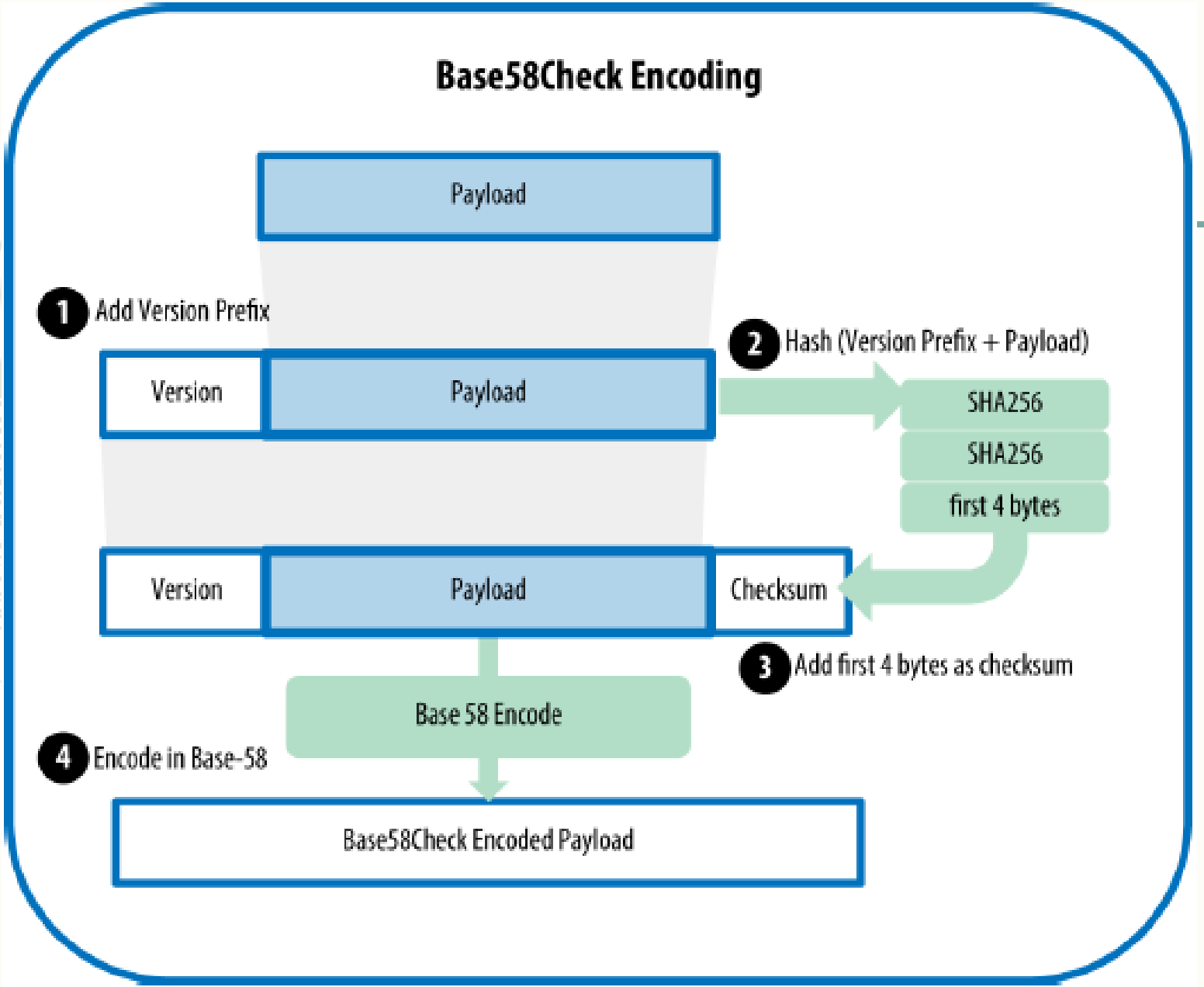


0x80 in hex

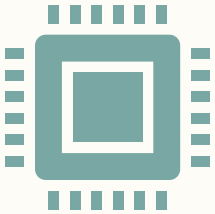
128 in decimal

the prefix used when encoding a private key

Illustration of Base58 Encoding

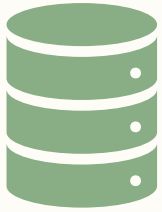


Conversion to Base58Check



Next

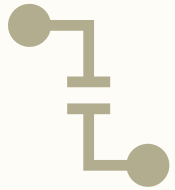
compute the “double-SHA” checksum



meaning

apply the SHA256 hash algorithm twice

on the previous result (prefix and data)



Formula

checksum =
SHA256(SHA256(prefix+data))

Conversion to Base58Check



Result

32-byte hash (hash-of-a-hash),

take only the first four bytes



Checksum

four bytes serve as the error-checking code, or checksum



Concatenate

The checksum is concatenated (appended) to the end.

Result

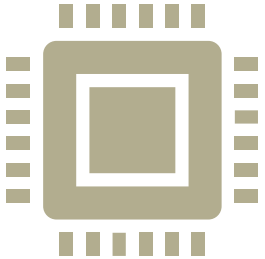


Composition of the result

a prefix

the data

checksum



Encode in Base58

This result is encoded using the Base58 alphabet

Table 4-1. Base58Check version prefix and encoded result examples

Type	Version prefix (hex)	Base58 result prefix
Bitcoin Address	0x00	1
Pay-to-Script-Hash Address	0x05	3
Bitcoin Testnet Address	0x6F	m or n

Type	Version prefix (hex)	Base58 result prefix
Private Key WIF	0x80	5, K, or L
BIP-38 Encrypted Private Key	0x0142	6P
BIP-32 Extended Public Key	0x0488B21E	xpub

Creating a bitcoin address from a private key

Base58Check-encoded bitcoin address

Key Formats



Key Representation

Both private

public keys



can be represented

in a number of

different formats



Values are the same

even though they look different

all encode the same number

Key Formats



Readibility

Main reason of new formats



Representation formats

are primarily used

to read and transcribe keys easily



Error preventions

without introducing errors

Private key formats



The private key

256-bit number



can be represented

in a number of different formats,



All are the same

correspond to

the same 256-bit number

Private key formats



Usage of formats

are used

in different circumstances.



Raw & Hexadecimal

used internally in software

rarely shown to users



WIF

for import/export of private keys

between wallets



QR code

often used in QR cod/barcode

Table 4-2. Private key representations (encoding formats)

Type	Prefix	Description
Raw	None	32 bytes
Hex	None	64 hexadecimal digits
WIF	5	Base58Check encoding: Base58 with version prefix of 128- and 32-bit checksum
WIF-compressed	K or L	As above, with added suffix 0x01 before encoding

Private keys: 3 common formats

Table 4-3. Example: Same key, different formats

Format	Private key
Hex	1e99423a4ed27608a15a2616a2b0e9e52ced330ac530edcc32c8ffc6a526aedd
WIF	5J3mBbAH58CpQ3Y5RNJpUKPE62SQ5tfcvU2JpbnkeyhfsYB1Jcn
WIF-compressed	KxFC1jmwwCoACiCAWZ3eXa96mBM6tb3TYzGmf6YwgdGWZgawvrtJ

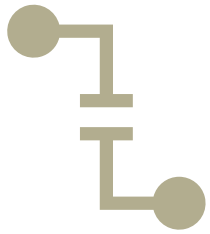
Decode from Base58Check



The Bitcoin Explorer

commands make it easy

to write shell scripts command-line “pipes”



Manipulation

bitcoin keys

addresses

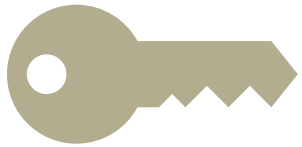
and transactions

Decode from Base58Check



base58check-decode

to decode the Base58Check format on the command line.



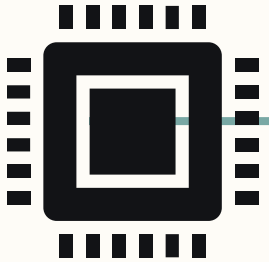
The result

contains the key as payload,

the WIF version prefix 128

a checksum.

Compression





Sufix 01

the “payload” of the compressed key is

appended with the suffix 01

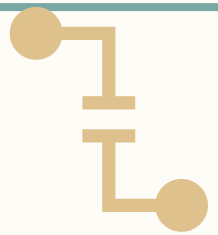
Compressed public key

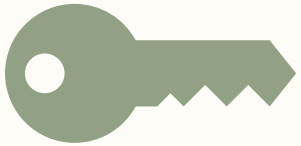
Signaling the derived public key

İs to be compressed

```
$ bx base58check-decode KxFC1jmwwCoACiCAWZ3eXa96mBM6tb3TYzGmf6YwgdGWZgawvrtJ
wrapper
{
        checksum 2339607926
        payload 1e99423a4ed27608a15a2616a2b0e9e52ced330ac530edcc32c8ffc6a526aedd01
        version 128
}
```

Encode from hex to Base58Check





base58check-encode

Opposite of decode

To encode into Base58Check

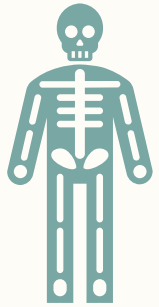
provide

the hex private key

followed by the WIF version prefix 128

bx base58check-encode
1e99423a4ed27608a15a2616a2b0e9e52ced330ac530edcc32c8ffc6a526aedd --version 128
5J3mBbAH58CpQ3Y5RNJpUKPE62SQ5tfcvU2JpbnkeyhfsYB1Jcn

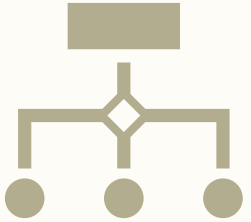
Encode from hex (compressed key) to Base58Check



Base58Check

To encode into Base58Check

as a “compressed” private key



Append

the suffix **01** to the hex key

then encode as in the preceding section

$ bx base58check-encode
1e99423a4ed27608a15a2616a2b0e9e52ced330ac530edcc32c8ffc6a526aedd01 --version 128
KxFC1jmwwCoACiCAWZ3eXa96mBM6tb3TYzGmf6YwgdGWZgawvrtJ

Encode from hex (compressed key) to Base58Check



“K”

Format starts with a “K”



denotes

the private key within

has a suffix of “01”



will be used

to produce compressed

public keys only

$ bx base58check-encode
1e99423a4ed27608a15a2616a2b0e9e52ced330ac530edcc32c8ffc6a526aedd01 --version 128
KxFC1jmwwCoACiCAWZ3eXa96mBM6tb3TYzGmf6YwgdGWZgawvrtJ

Public key formats



Public key types

compressed

Uncompressed



What is a Public key

a point

on elliptic curve



consisting of

a pair of coordinates

(x,y)

Public key formats



(x,y)

the prefix 04

followed by (x,y)



x

256-bit number



y

256-bit number

Public key formats



uncompressed

has prefix 04



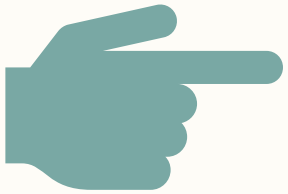
compressed

begin with

a 02

or a 03

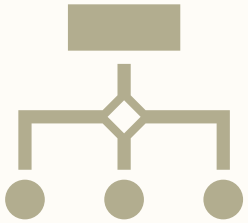
Uncompressed Public Key



Public key

520-bit number

130 hex digits



04 x y

the prefix 04

followed by x

then y coordinates

Uncompressed public key



Coordinates

(x,y)



x =

F028892BAD7ED57D2FB57BF33
081D5CFCF6F9ED3D3D7F159C2
E2FFF579DC341A



y

07CF33DA18BD734C600B96A72
BBC4749D5141C90EC8AC328AE
52DDFE2E505BDB

K =
04F028892BAD7ED57D2FB57BF33081D5CFCF6F9ED3D3D7F159C2E2FFF579DC341A07CF33DA18BD734C600B96A72BB
C4749D5141C90EC8AC328AE52DDFE2E505BDB

Public Key Compression



the curve

expresses a mathematical function



a point

on the curve

represents a solution to the equation



calculate the *y* coordinate

solving the equation

$y^2 \bmod p = (x^3 + 7) \bmod p$.

Public Key Compression



Store

only the *x* coordinate of the public key point



omitting the *y* coordinate

reducing the size of the key

and the space required to store it by 256 bits



50% reduction

An almost 50% reduction in size

in every transaction

02 & 03 Prefixes



04 prefix

uncompressed public keys



Prefixes 02 & 03

compressed public keys



02

a positive

Prime order p: even



03

negative value

Prime order p: odd

02 & 03 Prefixes

$y^2 = (x^3 + 7)$ over $(\mathbb{F}_p)$ — $y^2 \mod p = (x^3 + 7) \mod p$





Sign of y

Store just the sign of y

the left side of the equation is y^2

Square root

the solution for *y*

is a square root

Public Key Compression



Reduction

introduced to bitcoin

to reduce the size of transactions



Disk space

conserve disk space on nodes

that store the bitcoin blockchain database



Public key in transaction

Most transactions include the public key,

Public Key Compression



Need

required to validate the owner's credentials

and spend the bitcoin



520 bits

Per transaction

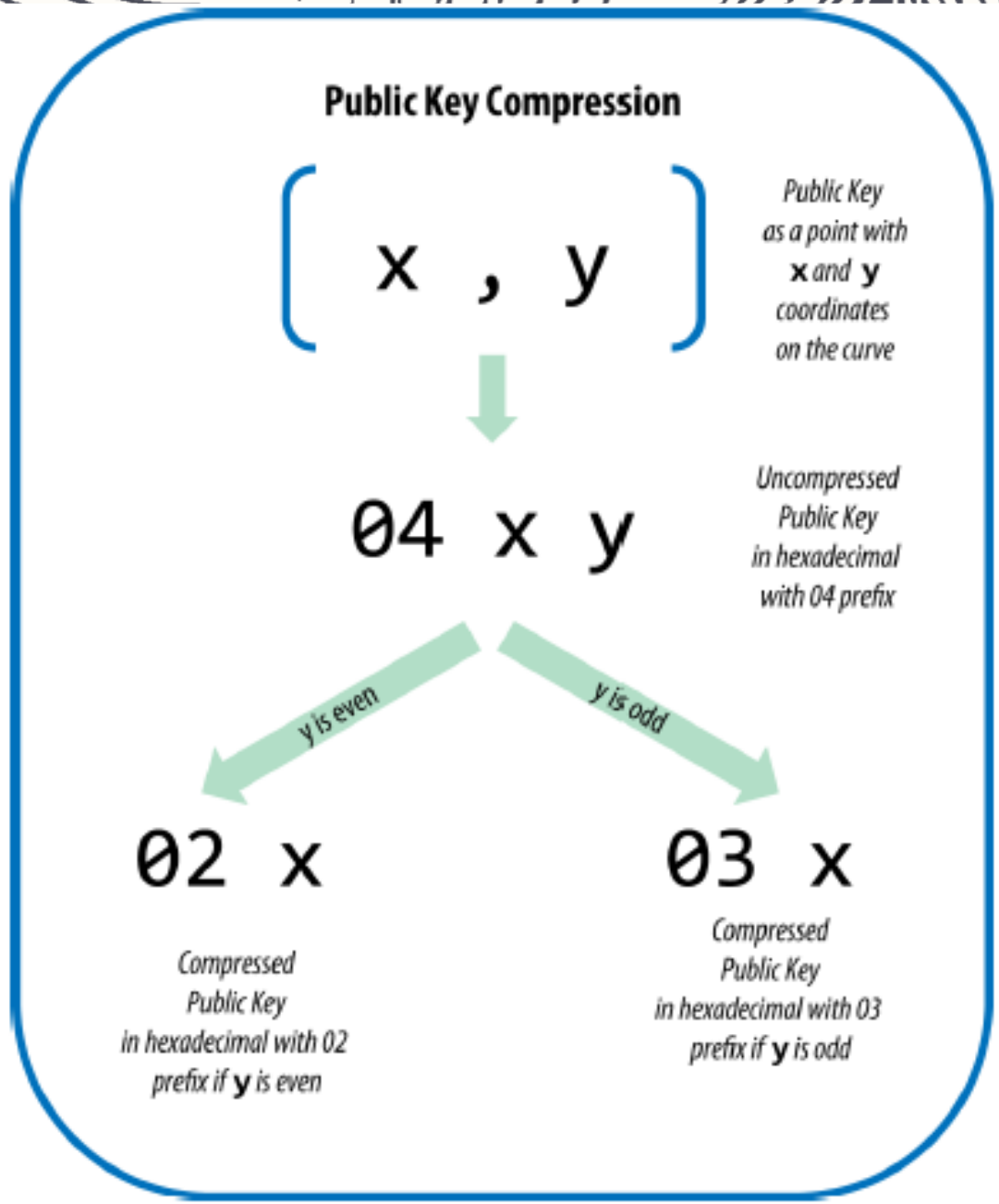
public key with (prefix + x + y),



data

100s of transactions per block

10 000's of transactions per day


Public Key Compression
[x , y]
Public Key as a point with x and y coordinates on the curve
04 x y
Uncompressed Public Key in hexadecimal with 04 prefix
y is even
y is odd
02 x
Compressed Public Key in hexadecimal with 02 prefix if y is even
03 x
Compressed Public Key in hexadecimal with 03 prefix if y is odd

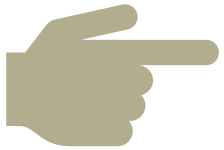
Example



Compressed Public Key

stored in

264 bits/66 hex digits



prefix 03

indicating

the *y* coordinate is odd



K

03F028892BAD7ED57D2FB57BF3308
1D5CFCF6F9ED3D3D7F159C2E2FFF5
79DC341A

Two Representation of the same Private Key



Visualization of Key

Compressed and uncompressed public key of the private key

Looks different from each other



Converting

a bitcoin address can be converted

using the double-hash function



Command

RIPEMD160(SHA256(K))

it will produce a *different* bitcoin address

Two Representation of the same Private Key



Confusing

Identical private key

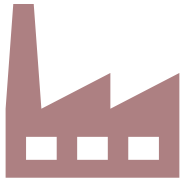
can produce a public key



expressed in two different formats

Compressed

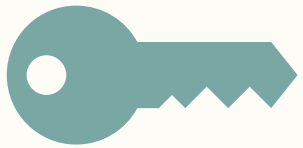
uncompressed



produce

two different bitcoin addresses

Client Support for Compression



Popularity

Compressed public keys are becoming

the default across bitcoin clients



Reason

a significant impact on reducing the size of transactions

and therefore the blockchain.

Client Support for Compression



Other clients

not all clients support compressed public keys yet



Newer clients

support compressed public keys

have to account for transactions

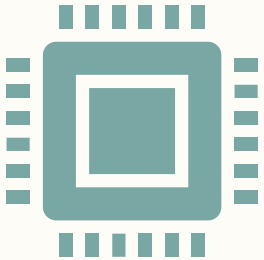


from older clients

do not support

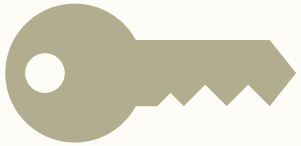
compressed public keys

Compressed private keys



WIF-compressed

“compressed private key”



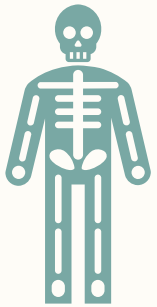
Means “private key

from which

only compressed public keys

should be derived”

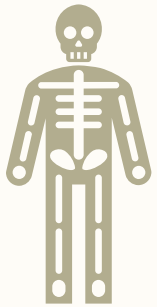
Compressed private keys



“WIF”: not “compressed”

“uncompressed private key”

really means



“private key

from which

only uncompressed public keys

should be derived.”

Format	Private key
Hex	1E99423A4ED27608A15A2616A2B0E9E52CED330AC530EDCC32C8FFC6A526AEDD
WIF	5J3mBbAH58CpQ3Y5RNJpUKPE62SQ5tfcvU2JpbnkeyhfsYB1Jcn
Hex-compressed	1E99423A4ED27608A15A2616A2B0E9E52CED330AC530EDCC32C8FFC6A526AEDD01
WIF-compressed	KxFC1jmwwCoACiCAWZ3eXa96mBM6tb3TYzGmf6YwgdGWZgawvrtJ

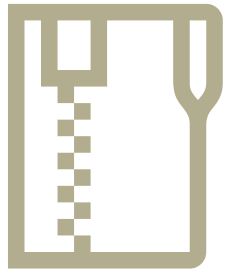
Table 4-4 shows the same key, encoded in WIF and WIF-compressed formats.

Strategies to Implement Wallet



Private keys -> Public keys

The private keys will be used
to derive the public key points on the curve



Wallet

Wallet will use
compressed public keys
in transactions

Strategies to Implement Wallet



Compressed public keys->Bitcoin addresses

The compressed public keys

will be used to produce bitcoin addresses



Wallet

Wallet will use bitcoin addresses

in transactions

Strategies to Implement Wallet



Exporting from a new wallet

implements compressed public keys



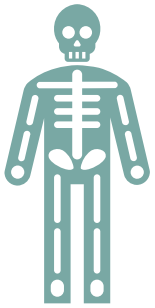
WIF Modification

the WIF is modified

With the addition of a one-byte suffix 01

to the private key.

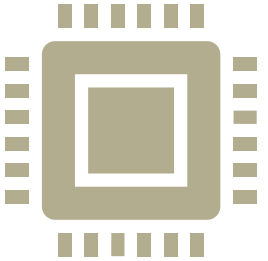
Strategies to Implement Wallet



Meaning of K* or *L

Private key starts with the letter *K* or *L*

“compressed WIF” Base58Check-encoded private



Meaning of “5”

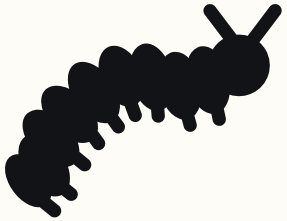
Private key starts with “5”

«noncompressed WIF-encoded



"Compressed private keys" is a misnomer! They are not compressed; rather, WIF-compressed signifies that the keys should only be used to derive compressed public keys and their corresponding bitcoin addresses. Ironically, a "WIF-compressed" encoded private key is one byte longer because it has the added `01` suffix to distinguish it from an "uncompressed" one.

Implementing Keys and Addresses in Python



pybitcointools

A clone of Vitalik Buterin bitcoin library



pybitcointools library

imported as “bitcoin”

to generate and display keys and addresses in various formats.

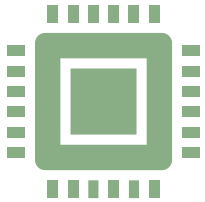
pip install cryptos

uses of os.urandom



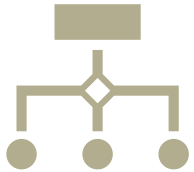
CSRNG

a cryptographically secure random number generator



Provided

provided by the underlying operating system.



Unix-like operating system such as Linux,

it draws from /dev/urandom;

uses of os.urandom



Windows

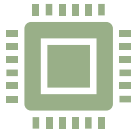
it calls CryptGenRandom().



NotImplementedError

Will be raised

if a suitable randomness source is not found



Security

random number generator used here is not appropriate

for generating production-quality bitcoin keys



Don’t use it

as it is not implemented

with sufficient security.

Advanced Keys and Addresses



Encrypted Private Keys



Pay-to-Script Hash (P2SH)
& Multisig Addresses



Vanity Addresses



Paper Wallets

Encrypted Private Keys



confidentiality

Private keys must remain secret



Confidentiality vs availability.

Confidentiality is quite difficult to achieve in practice

because it conflicts with the equally



Back up

private key to avoid losing it

Confidentiality is much harder



Move of Private Keys

From one wallet to another

Store on paper

Store on external media

BIP-38



Confidentiality of Backups and Copes

what will happen if the backup itself is stolen or lost?



BIP - 38

Standard for encrypting private keys

portable and convenient



Benefit

can be understood

by many different wallets and bitcoin clients

BIP-38 Proposes



Passphrase

a common standard for encrypting private keys

with a passphrase



Protection

against exposure

encoding private keys with Base58Check



Secure storage

stored securely on backup media,

transported securely between wallets

kept in any other conditions

How it Works?



AES

Advanced Encryption Standard

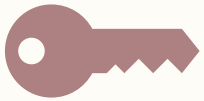
The standard for encryption uses established by the NIST



Use cases

data encryption implementations

for commercial and military applications



BIP-38

encryption scheme

How it Works?



input

a bitcoin private key



encoded

in the WIF

as a Base58Check

string with the prefix of “5.”



password

BIP-38 encryption scheme

takes a passphrase



passphrase

composed of several words

or a complex alphanumeric string

How it Works?



The result BIP-38 encryption scheme

is a Base58Check-encoded encrypted private key

begins with the prefix 6P.



a key that starts with 6P

is encrypted

and requires a passphrase in order to convert (decrypt)



Use in wallet

it back into a WIF-formatted private key (prefix 5)

that can be used in any wallet.

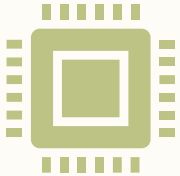
How it Works?



Recognition

Many wallet applications now recognize

BIP-38-encrypted private keys



Prompt

will prompt the user for a passphrase

to decrypt and import the key



How to decrypt BIP-38 keys

Third-party applications,

such as the incredibly useful browser-based Bit Address (Wallet Details tab)

Table 4-5. Example of BIP-38 encrypted private key

Private Key (WIF)	5J3mBbAH58CpQ3Y5RNJpUKPE62SQ5tfcvU2JpbnkeyhfsYB1Jcn
Passphrase	MyTestPassphrase
Encrypted Key (BIP-38)	6PRTHL6mWa48xSopbU1cKrVjpKbBZxcLRRCdctLJ3z5yxE87MobKoXdTsJ

Pay-to-Script Hash (P2SH) & Multisig Addresses



traditional bitcoin addresses begin with “1”

are derived from the public key

which is derived from the private key.



Spending

Spending of bitcoin with "1" addresses

can only be spent by presenting the keys

Pay-to-Script Hash (P2SH) & Multisig Addresses



“3”

Bitcoin addresses that begin with the number are **pay-to-script hash** (P2SH) addresses

sometimes erroneously called «multisignature» or «multisig» addresses.

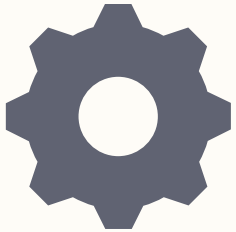


Hash of script transfer

They designate the beneficiary of a bitcoin transaction as the hash of a script

instead of the owner of a public key.

P2PKH – Pay-to-public-key-hash



History

The feature was introduced in January 2012 with BIP-16



Use Scenario

provides the opportunity

to add functionality to the address itself

P2PKH – Pay-to-public-key-hash

P2PKH

also known as a pay-to-public-key-hash

Stronger proof of ownership

require additional conditions

next to

public key hash

private key signature

Encoding a P2SH Address



involves

using the same double-hash function

as used during creation of a bitcoin address



difference

only applied on the script instead of the public key

script hash = RIPEMD160(SHA256(script))



Encoding resulting “script hash”

with Base58Check with a version prefix of 5

results in an encoded address starting with a 3

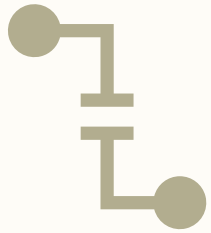
Example P2SH Address



Example

An example of a P2SH address

3F6i6kwkevjR7AsAd4te2YB2zZyASEm1HM



Deriving

can be derived using the Bitcoin Explorer commands

script-encode, sha256, ripemd160, and base58check-encode

Multisignature addresses and P2SH



multisignature address script

Currently the most common implementation of the P2SH function



Prove of ownership

Requires more than one signature

To prove ownership and spend funds



M-of-N multisig

The bitcoin multisignature feature is designed

to require M signatures from a total of N keys

M-of-N multisig where M =< N



1-of-2 signatures

Bob and his spouse owns 2 kyes

Any of them can sign and spend the fund

Vanity Addresses

Length	Pattern	Frequency	Average search time
1	1K	1 in 58 keys	< 1 milliseconds
2	1Ki	1 in 3,364	50 milliseconds
3	1Kid	1 in 195,000	< 2 seconds
4	1Kids	1 in 11 million	1 minute
5	1KidsC	1 in 656 million	1 hour
6	1KidsCh	1 in 38 billion	2 days
7	1KidsCha	1 in 2.2 trillion	3–4 months
8	1KidsChar	1 in 128 trillion	13–18 years
9	1KidsChari	1 in 7 quadrillion	800 years
10	1KidsCharit	1 in 400 quadrillion	46,000 years
11	1KidsCharity	1 in 23 quintillion	2.5 million years

Table 4-7. The frequency of a vanity pattern (1KidsCharity) and average search time on a desktop PC

Addresses with messages

- valid bitcoin addresses that contain human-readable messages.

«Love»

- 1LoveBPzzD72PUXLzCkYAtGFYmK5vYNR33 is a valid address
- contains the letters forming the word “Love” as the first four Base-58 letters

Trial&Error

- require generating and testing billions of candidate private keys
- until a bitcoin address with the desired pattern is found.

Outsource to Vanity Pool

Pattern length	Honest fee	Minimum fee
1-6	0.01	0.01
7	0.14	0.01
8	8.3	0.08
9	480	4.8
10	27862	278
11	1616015	16160

Outsource

- Another way to find a vanity address is
- to outsource the work to a pool of vanity miners, such as the pool at Vanity Pool.

pool of vanity miners

- A pool is a service
- to earn bitcoin searching for vanity addresses for others

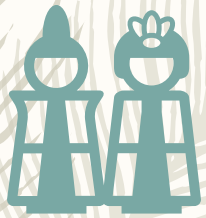
Cost

- 0.01 bitcoin (92,82USD)
- approximately $5 at the time

Time for 7-characte

- get results in a few hours
- instead of having to run a CPU search for months

Vanity address security



Improvment in security

a distinctive address makes it harder for adversaries

to substitute their own address and fool customers



3,364 times (58 × 58) higher effort

expended effort to match at least 6 characters (two more),

than the effort expended for 4-character vanity



Fooling customers

anyone to create an address that resembles or the same of vanity address,

thereby fooling your customers

Original Random Address	1J7mdg5rbQyUHENYdx39WVWK7fsLpEoXZy
Vanity (4-character match)	1J7md1QqU4LpctBetHS2ZoyLV5d6dShhEy
Vanity (5-character match)	1J7mdgYqyNd4ya3UEcq31Q7sqRMXw2XZ6n
Vanity (6-character match)	1J7mdg5WxGENmwyJP9xuGhG5KRzu99BBCX

Paper Wallets

Private keys on paper

the paper wallet

also includes

the corresponding bitcoin address

for convenience,

but this is not necessary

because it can be derived

from the private key.

“cold storage”

creates backups

or offline bitcoin storage

Can be generated offline

and never stored online

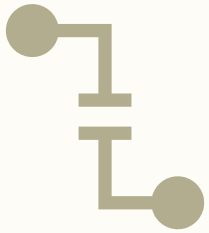
Public address	Private key (WIF)
1424C2F4bC9JidNjjTUZCbUxv6Sa1Mt62x	5J3mBbAH58CpQ3Y5RNJpUKPE62SQ5tfcvU2JpbnkeyhfsYB1Jcn

Paper Wallets



Security against threads

hackers, keyloggers, …



Protection against incidents

such as a hard-drive failure, theft, or accidental deletion

Public address	Private key (WIF)
1424C2F4bC9JidNjjTUZCbUxv6Sa1Mt62x	5J3mBbAH58CpQ3Y5RNJpUKPE62SQ5tfcvU2JpbnkeyhfsYB1Jcn

How to Generate Paper Wallets?

Generator

using a tool

such as the client-side JavaScript generator

at bitaddress.org.

This page contains

all the code

necessary to generate keys and paper wallets

even while

completely disconnected

from the internet.



How to Generate Paper Wallets?



How to use?



save the HTML page

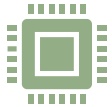
on your local drive

or on an external USB flash drive



Disconnect from the internet

open the file in a browser.



Even better

boot your computer using a pristine operating system,



such as

a CD-ROM bootable Linux OS.



print on a local printer

over a USB cable

not wirelessly

Protection against Thief



Access to the paper



A thief

who is able

to gain access to the paper



can either steal it

or photograph the keys



take control of the bitcoin

locked with those keys

Protection against Thief



BIP-38 encrypted private keys



the paper wallet

are protected by a passphrase



Without the passphrase

the encrypted keys

are useless.

examples of paper wallets with security and backup features

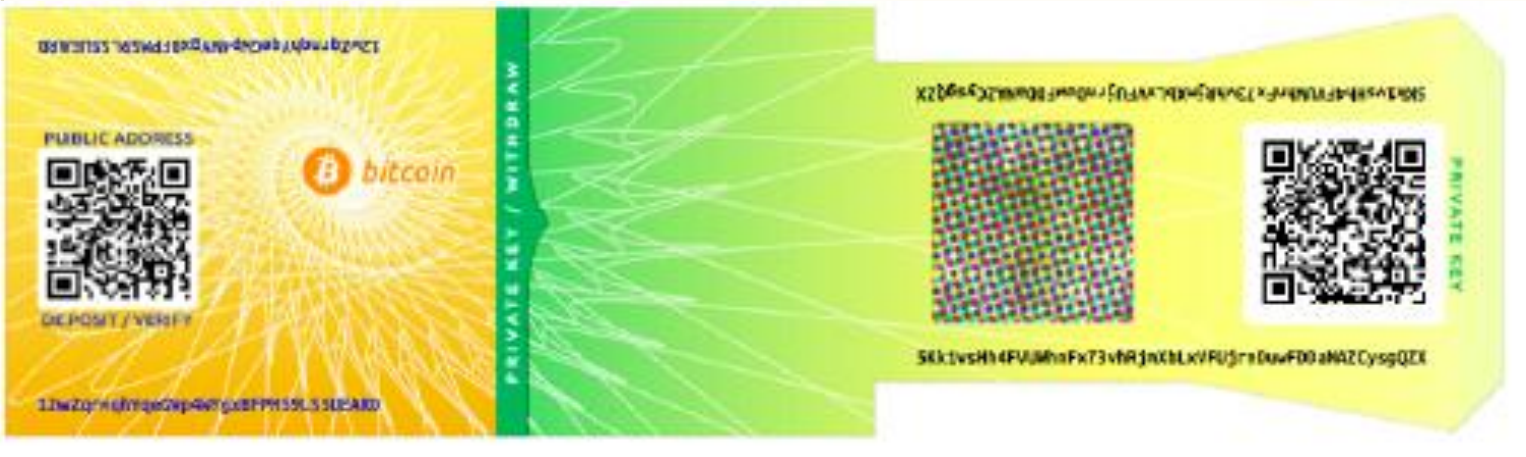


Figure 4-10. An example of a paper wallet from bitcoinpaperwallet.com with the private key on a folding flap



Figure 4-11. The bitcoinpaperwallet.com paper wallet with the private key concealed



Figure 4-12. An example of a paper wallet with additional copies of the keys on a backup "stub"

Discussions

Your opinion

What do you think on this chapter?

Challenging

What is the most interesting point?

Clarification

What are the unclarified points?

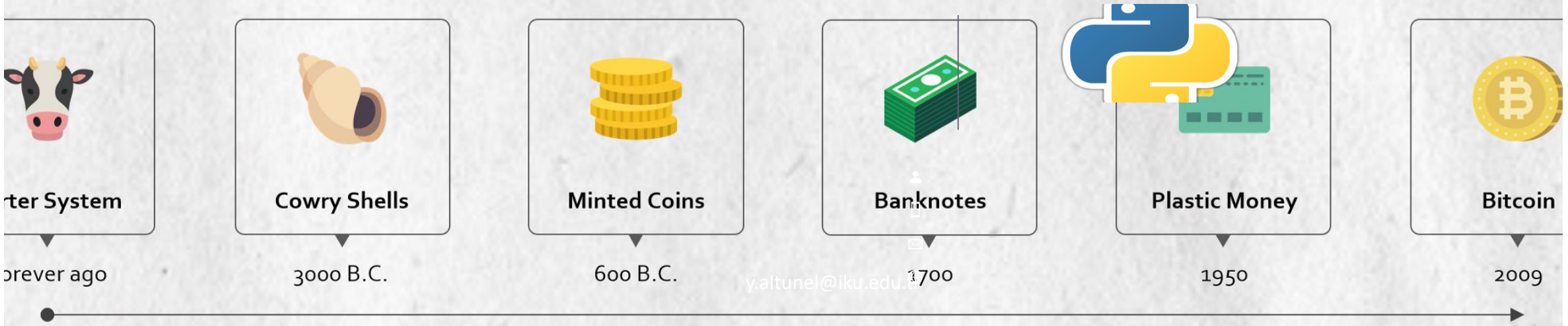
Critical issue

Select and explain a critical issu in this chapter



Thank You

The History of Money



Yusuf Altunel, PhD

https://www.linkedin.com/in/yusuf-altunel-2070a31b/

https://cats.iku.edu.tr/portal/site/cd88fc16-3584-46f0-a539-890caf466d53